Case-Based Reasoning in Design

Mary Lou Maher and Andrés Gómez de Silva Garza, University of Sydney

CASE-BASED REASONING IS A GENeral paradigm for problem solving based on the recall and reuse of specific experiences. AI practitioners have applied and extended this paradigm for numerous classes of design problems, indicating that it is an important aspect of design support. This survey considers the range and types of design applications that have implemented case-based reasoning and shows how these applications have resolved theoretical issues through their implementations.

THIS SURVEY PRESENTS A NUMBER OF ISSUES RAISED BY THE APPLICATION OF CASE-BASED REASONING FOR DESIGN. THE AUTHORS COMMENT ON THE TWO EXTREMES OF CASE-BASED DESIGN: DESIGN ASSISTANCE AND DESIGN AUTOMATION, AND HOW THEY SHARE A NUMBER OF DIFFICULT ISSUES RELATED TO THEIR IMPLEMENTATION.

Learning to design

CBR supports design by reminding designers of previous experiences that can help with new situations. As designers, we learn to design by experiencing design situations. This is also how we teach students to be designers. For example, engineers learn to have analytical capabilities in school and then learn to design in professional practice; architects learn through a range of design experiences in the studio. As designers, we learn to analyze through the use of formal methods, but creating a new design requires previous experience, or at least, exposure to another's design experiences. Current formal methods for generating designs are useful for programming computers to design, but not necessarily as models of human thinking. In CBR, there are similarities between the formal method implemented in a computer program and the informal observation that human designers rely heavily on specific design experiences.

CBR's problem-solving approach relies on analogy, where problems or experiences outside the one we are currently addressing might provide some insight or assistance. Through analogy, we might be reminded of a window design when designing a door to a balcony with a view. Analogy lets us recognize something that we have not encountered before, by associating it with something we have.

Considering analogy from the perspective of memory and reminding has led to the concept of memory organization as a guideline for computer representations.[1] The study of memory organization, and its use as a basis for solving new problems and generating explanations, has led to models for representing experience in computers.[2] This AI research area has found use in real-world situations.[3,4]

CBR formalizes a computational model of problem solving based on memory organization and reminding. In addition to addressing memory organization, CBR provides a process model with specific stages. CBR lets us implement computer support for designers by applying memory-organization research to define a case memory of design experience and by applying recall and adaptation processes to reuse the experience.

Both CBR and expert systems rely on the explicit symbolic representation of experience-based knowledge to solve a new problem. However, expert systems use past expe-

0885-9000/97/$10.00 © 1997 IEEE

设计中的基于案例的推理

玛丽·卢·马赫尔与安德烈斯·戈麦斯·德·席尔瓦·加尔萨，悉尼大学

基于案例的推理是一种通用的问题求解范式，其核心在于对具体经验的回忆与重用。人工智能领域的从业者已将这一范式应用于并拓展至众多类型的设计问题，表明其在设计支持中具有重要地位。本综述考察了已实现基于案例推理的设计应用的范围与类型，并阐述了这些应用如何通过具体的实现来解决相关的理论问题。

THIS SURVEY PRESENTS A NUMBER OF ISSUES RAISED BY THE APPLICATION OF CASE-BASED REASONING FOR DESIGN. THE AUTHORS COMMENT ON THE TWO EXTREMES OF CASE-BASED DESIGN: DESIGN ASSISTANCE AND DESIGN AUTOMATION, AND HOW THEY SHARE A NUMBER OF DIFFICULT ISSUES RELATED TO THEIR IMPLEMENTATION.

学习设计

案例推理通过提醒设计者回顾以往的经验，从而帮助其应对新的情境。作为设计者，我们是在不断经历设计实践的过程中学习如何进行设计的，这也是我们教导学生如何成为设计者的方式。例如，工程师在学校中培养分析能力，随后在专业实践中逐步掌握设计技能；建筑师则通过工作室中的多种设计实践积累经验。作为设计者，我们借助形式化方法来提升分析能力，但要创造出新的设计方案，仍需依赖既往的设计经验，或至少接触他人的设计实践。当前用于生成设计方案的形式化方法在……方面颇具价值。通过编程使计算机能够进行设计，但并不一定将其作为人类思维的模型。在基于案例的推理中，计算机程序所实现的形式化方法与人类设计师高度依赖特定设计经验这一非正式观察之间存在相似之处。

案例推理的问题求解方法依赖于类比，即在当前所处理的问题之外的其他问题或经验可能提供某种启发或帮助。通过类比，在设计一处观景阳台的门时，我们可能会联想到一种窗户的设计方案。类比使我们能够将未曾遇到的事物与已有的知识相联系，从而识别出其相似之处。

从记忆与提示的角度进行类比，已促使人们提出了将记忆组织作为计算机表示方法指导原则的概念。[1] 对……的研究记忆组织及其作为解决新问题与生成解释的基础，已催生了用于在计算机中表征经验的模型。[2] 该人工智能研究领域已在实际应用中得到广泛运用。[3,4]

案例推理（CBR）形式化了一种基于记忆组织与提醒的问题求解计算模型。除解决记忆组织问题外，CBR还提供了一个包含特定阶段的过程模型。通过将记忆组织研究应用于设计经验的案例库构建，并将回忆与适应过程应用于经验的再利用，CBR使我们能够为设计人员提供计算机辅助支持。

案例推理与专家系统均依赖于基于经验的知识的显式符号化表示来解决新问题。然而，专家系统则利用以往的经验……

0885-9000/97/$10.00 © 1997 IEEE

rience stored in a knowledge base of generalized heuristics to assist in solving a new problem.[5] They can store the generalized heuristics as rules of thumb or as logical inferences. CBR uses an abstraction of specific problem-solving experiences to learn to solve a new problem. The representation of specific experiences usually includes the justification of the solution or the requirements of the problem as well as its solution.

Two major considerations in a CBR approach to design are

- representations of design cases and
- process models for recalling and adapting design cases.

A high-level view of CBR illustrates the role of design cases in the CBR process (see Figure 1). Representing design cases requires an abstraction of the experience into a symbolic form that the reasoner (either computer or human) can manipulate. As this survey shows, practitioners have developed various abstractions based on models, methods, and philosophical approaches to understanding design, designers, and design experiences.

Design-case recall involves finding a *relevant* design experience; it decomposes into the subtasks of *indexing*, *retrieval*, and *selection* (see Figure 2a). Indexing identifies the features a previous design should have to be relevant to the new design problem. Retrieval identifies which cases have all or a subset of those features, and selection evaluates the retrieved cases so they can be ranked.

Design-case adaptation recognizes the differences between the selected design case and the new design problem, and changes the design case so that it solves the new design problem. This process decomposes into three steps: *propose*, *evaluate*, and *modify* (see Figure 2b). Before starting on a modify-evaluate cycle, the process involves proposing the selected design case as a solution to the new design problem. Evaluation checks the feasibility of the new design solution. Modification changes parts of the new design solution, thus transforming it into a different proposed solution that must then be evaluated.

Applications of CBR to design

Several recent publications have addressed specific considerations of the application of CBR to design.[6–8] These applications have ranged from largely informal domains, such as meal planning, to analytical domains, such as the design of mechanical devices. Each design application brings a new set of considerations to the use of CBR in design. The application of CBR to design has produced a number of implementations that resolve theoretical issues. Each implementation serves as an example for those who intend to use case-based design as a computer support system for designers. Table 1 lists a selection of implementations that serve as the basis for our discussion. The domains shown reflect the range of issues introduced by the need to develop practical applications.

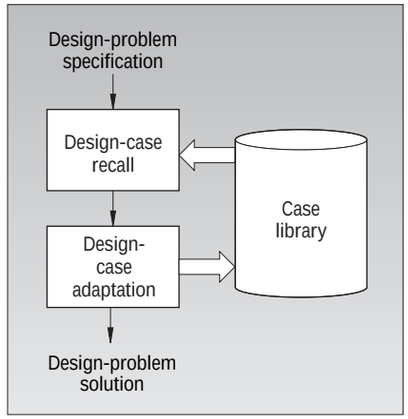


Figure 1. The overall process model of case-based design.

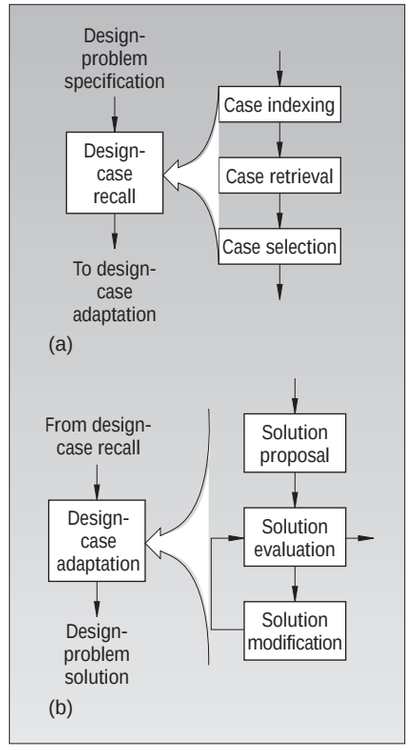


Figure 2. Generic processes: (a) case recall; and (b) case adaptation.

We consider these recurring themes in the implementation of case-based design:

- the need to represent and manage *complex design cases*,
- the need to augment a set of specific experiences with *generalized design knowledge*, and
- the need to formalize a typically *informal body of knowledge* or experiences.

Complex design cases. Design in any domain usually involves the development and understanding of complex systems. The complex representations needed to adequately capture a design case have introduced challenges to the implementation of case-based design. The CBR paradigm assumes that there is a concept of "a case"; in most design domains, however, a design is not simply "a case" but a complex set of experiences and decisions resulting in a complex system. Research has addressed this complexity issue in three ways:

- representing a design case as a *partonomic hierarchy*,
- representing *different views* of the same design case, and
- using *multimedia* representations for different types of information.

These three approaches are not exclusive; one implementation can use more than one approach.

A partonomic hierarchy of a design case is a decomposition of the design experience into a hierarchy of parts or subcases. This decomposition allows reasoning to focus only on the relevant parts of a design. Processing only some of the knowledge associated with a case lets reasoning become more efficient. The development of a hierarchically structured case base requires defining a typical decomposition of a design experience. System designers might represent this decomposition explicitly through design models, or use it implicitly for case acquisition.

Using a partonomic hierarchy is the most common way to address design complexity. System developers used this approach, with slight variations, in Casecad, Cadsyn, Gencad, Panda, Asp-II, and Benton. For example, Casecad and Cadsyn represent subcases with the *Function Behavior Structure* (FBS) framework. This framework classifies the variables used to describe design cases into three semantically different categories. *Function variables* describe aspects of a design related to its purposes and the designer's

经验被存储于一个包含广义启发式规则的知识库中，以协助解决新问题。这些启发式规则既可以以经验法则的形式存储，也可以以逻辑推理的形式存储。案例推理则通过对具体问题求解经验的抽象，来学习如何解决新问题。具体经验的表示通常包括解决方案的合理性依据、问题的需求以及相应的解决方案。

在基于案例的推理设计方法中，有两个主要考量因素：

- 设计案例的表示形式以及
- 用于检索与适应设计案例的过程模型。

CBR的高层次视图展示了设计案例在CBR过程中的作用（见图1）。设计案例的表示需要将经验抽象为一种符号化形式，以便推理者（无论是计算机还是人类）能够对其进行操作。正如本综述所表明的那样，实践者已基于对设计、设计师及设计经验的理解，结合模型、方法以及哲学视角，发展出了多种抽象表示方法。

设计案例检索涉及寻找一个*rel*-设计经验；该过程可分解为*indexing*、*retrieval*和*selec*-三个子任务，详见图2a)。索引阶段用于识别先前设计应具备的、与当前设计问题相关的特征；检索阶段用于确定哪些案例包含全部或部分这些特征；而选择阶段则对检索到的案例进行评估，以便对其进行排序。

设计案例适配识别所选设计案例与新设计问题之间的差异，并对设计案例进行修改，使其能够解决新的设计问题。该过程可分为三个步骤：*propose*、*evaluate*和*modify*(，参见图2b)。在进入修改—评估循环之前，该过程首先将所选设计案例作为新设计问题的解决方案进行提出。评估环节用于检验新设计方案的可行性。修改环节则对新设计方案的部分内容进行调整，从而将其转化为另一种备选方案，随后该备选方案还需再次接受评估。

基于案例的推理在设计中的应用

近年来，已有数篇文献探讨了将案例推理应用于设计时需特别考虑的若干问题。这些应用已经

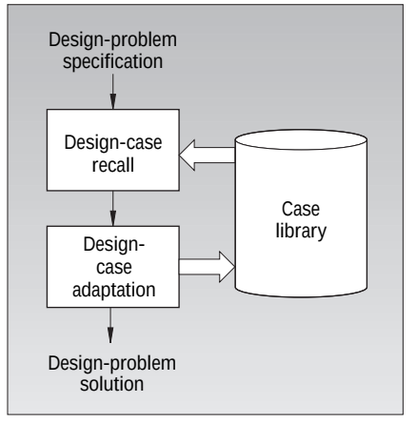


图1. 基于案例的设计总体流程模型。

其应用范围涵盖了从以非正式为主的领域（如膳食计划）到分析性较强的领域（如机械装置设计）。每一种设计应用场景都会为CBR在设计中的应用带来一系列新的考量因素。CBR在设计领域的应用已催生出若干实现方案，这些方案有效解决了相关理论问题。每一个实现都可作为案例，为拟将基于案例的设计方法用作设计师计算机辅助系统的研究者提供参考。表1列出了若干具有代表性的实现方案，它们构成了我们讨论的基础；所涉及的应用领域也反映了在开发实用化应用过程中所面临的各类问题。

我们探讨了在基于案例的设计实施过程中反复出现的若干主题：

- 表示和管理 *com-* 的需求，
- 需要将一组特定的经验与 *generalized design*进行扩充，并
- 有必要将通常基于*informal*或经验的方法加以规范化。

复杂的设计案例。任何领域的设计通常都涉及复杂系统的开发与理解。为充分刻画设计案例而所需采用的复杂表示方式，给基于案例的设计实现带来了诸多挑战。CBR范式假定存在“案例”这一概念；然而，在大多数设计领域中，一项设计并不仅仅是“一个案例”，而是一组由众多经验与决策共同构成、最终形成复杂系统的结果。研究已从三个方面着手应对这一复杂性问题：

- 将一个设计案例表示为 *parto-*，
- 代表同一设计案例的*different views*，且

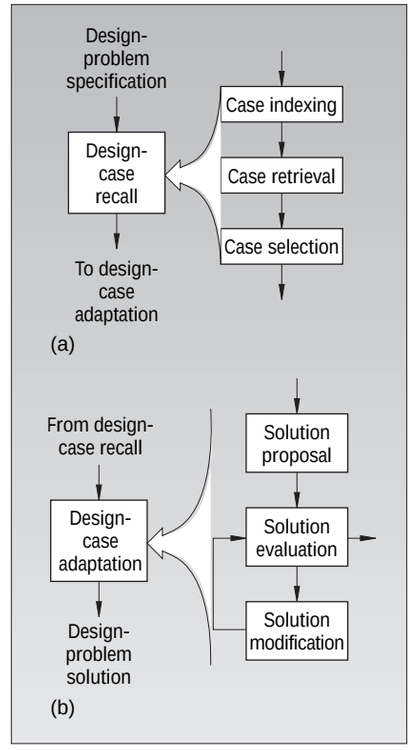


图2. 通用流程：（a）案例检索；（b）案例适配。

- 使用针对不同类型信息的*multimedia*表示法。

这三种方法并不互斥；一种实现可以同时采用多种方法。

设计实例的分层结构是将设计经验分解为由若干部件或子实例构成的层次体系。这种分解使得推理过程能够仅聚焦于设计中的相关部分，从而通过仅处理与实例相关的部分知识来提高推理效率。构建具有层次结构的案例库需要明确界定设计经验的典型分解方式。系统设计者可以通过设计模型对该分解进行显式表达，也可在案例获取过程中将其作为隐含依据加以利用。

采用分层树状结构是应对设计复杂性的最常用方法。系统开发人员在 Caseca d、Cadsyn、Gen-cad、Panda、Asp-II 和 Benton 等工具中均采用了这一方法，仅在具体实现上略有差异。例如，Casecad 和 Cadsyn 通过
Function Behavior Structure (FBS) 框架来表示子案例。该框架将用于描述设计案例的变量划分为三个语义上不同的类别。*Func-* 描述与设计目的及设计者相关的方面。

Table 1. Applications of CBR to design.

System name	Design domain	Reference
Archie, Archie-II	Architecture	E. Domeshek and J. Kolodner, "The Designer's Muse," in *Issues and Applications of Case-Based Reasoning in Design*, M.L. Maher and P. Pu, eds., Lawrence Erlbaum Associates, Hillsdale, N.J., 1997, pp. 11–38.
AskJef	Software	J. Barber et al., "AskJef: Integration of Case-Based and Multimedia Technologies for Interface Design Support," in *Artificial Intelligence in Design*, Kluwer Academic Publishers, Dordrecht, The Netherlands, pp. 457–475.
Asp-II, Benton	Software	C. Tsatsoulis and P. Alexander, "Integrating Cases, Sub-Cases, and Generic Prototypes for Design," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 261–300.
Cadet	Mechanical devices	S. Narashiman, K. Sycara, and D. Navin-Chandra, "Representation and Synthesis of Non-Monotonic Mechanical Devices," *Issues and Applications of Case-Based Reasoning in Design*, pp. 187–220.
Cadre, Faming	Architecture	B. Faltings, "Case Reuse by Model-Based Interpretation," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 39–60.
Cadsyn, Casecad	Structural systems	M.L. Maher, B. Balachandran, and D.M. Zhang, *Case-Based Reasoning in Design*, Lawrence Erlbaum, 1995.
Fabel	Architecture	A. Voss, "Case Design Specialists in Fabel," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 301–338.
Gencad	Structural systems	A. Gómez de Silva Garza and M.L. Maher, "The Adaptation of Structural System Designs Using Genetic Algorithms," *Proc. Int'l Conf. Information Technology in Civil and Structural Engineering Design*, Elsevier Science, New York, 1996, pp. 189–196.
Janus	Architecture	G. Fisher, R. McCall, and A. Morch, "Design Environments for Constructive and Argumentative Design," *Proc. Human Factors in Computing Systems Conf.*, ACM Press, New York, 1989, pp. 269–275.
Julia	Meal planning	T.R. Hinrichs, "Towards an Architecture for Open World Problem Solving," *Proc. CBR Workshop*, Morgan Kaufmann, San Francisco, 1988, pp. 182–189.
Kritik, Kritik II	Physical devices	A. Goel, S. Bhatta, and E. Stroulia, "KRITIK: An Early Case-Based Design System," *Issues and Applications of Case-Based Reasoning in Design*, pp. 87–132.
Panda	Fire engines	C. Tsatsoulis and P. Alexander, "Integrating Cases, Sub-Cases, and Generic Prototypes for Design," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 261–300.
Seed	Architecture	U. Flemming et al., "Case-Based Design in a Software Environment That Supports the Early Phases," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 61–86.

intentions. *Behavior variables* are features of a design description that concern the design's interaction with and response to its environment. *Structure variables* describe the design's physical, geometric, and visual properties. Gencad adds the notion of *context variables*, which describe features of the design's environment that might influence design decisions.

Fabel addresses the complexity of the building-design domain by using different views or aspects of a specific design. The Fabel system can represent a single design problem as several design cases, where each case is a different perspective on the design. This approach involves using different views of the design, such as *gestalts* (simplified holistic abstractions of a complex geometry), *attribute vectors* (lists of values of predetermined features), and *images* (digital images of the design solution). This approach distinguishes itself from the partonomic hierarchy by treating each view as a case and not necessarily relating all views of a case to make up a "whole" case description.

The multimedia approach treats complexity by organizing the information about a design around different storage formats, then cross-referencing the different representations. For example, in Archie-II, a design can consist of a building's photograph, a drawing of its architectural plan, and textual annotations about different aspects of the building's design. AskJef contains symbolic, graphical, and auditory representations for cases. AskJef also decomposes cases temporally, so that each case is actually a collection of different versions of a design that were generated iteratively during the design phase (only the last one of which was deemed to be a good solution).

In summary, designers address complexity by breaking a case into smaller, less complex abstractions. Table 2 summarizes the approaches used for breaking down a building design case, with example systems cited (the list of example systems is not exhaustive—other systems mentioned in this survey are not cited in Table 2, and many systems not mentioned here might also follow each approach).

Generalized design knowledge. The application of CBR to design domains that involve mechanical and other physical devices has extended the notion of design cases to include a representation of general knowledge in the form of causal or heuristic knowledge. In such design domains, the representation and reuse of design cases is impossible without some generalized knowledge. Using the terminology of the authors of case-based design systems, some methods for representing generalized design knowledge include

- causal models,
- state interactions,
- heuristic models,
- heuristic rules, and
- geometric constraints.

Kritik's developers directly addressed the representation of causal behavior through the use of *structure-behavior-function* models. (The SBF formalism uses the same words as the FBS formalism, but with fundamentally different meanings.) Kritik represents behaviors as sets of device states and transitions linking the states; these behaviors explain how the physical device's structural elements

表1. CBR在设计中的应用。

System name	Design domain	Reference
Archie, Archie-II	Architecture	E. Domeshek and J. Kolodner, "The Designer's Muse," in Issues and Applications of Case-Based Reasoning in Design, M.L. Maher and P. Pu, eds., Lawrence Erlbaum Associates, Hillsdale, N.J., 1997, pp. 11–38.
AskJef	Software	J. Barber et al., "AskJef: Integration of Case-Based and Multimedia Technologies for Interface Design Support," in Artificial Intelligence in Design, Kluwer Academic Publishers, Dordrecht, The Netherlands, pp. 457–475.
Asp-II, Benton	Software	C. Tsatsoulis and P. Alexander, "Integrating Cases, Sub-Cases, and Generic Prototypes for Design," in Issues and Applications of Case-Based Reasoning in Design, pp. 261–300.
Cadet	Mechanical devices	S. Narashiman, K. Sycara, and D. Navin-Chandra, "Representation and Synthesis of Non-Monotonic Mechanical Devices," Issues and Applications of Case-Based Reasoning in Design, pp. 187–220.
Cadre, Faming	Architecture	B. Faltings, "Case Reuse by Model-Based Interpretation," in Issues and Applications of Case-Based Reasoning in Design, pp. 39–60.
Cadsyn, Casecad	Structural systems	M.L. Maher, B. Balachandran, and D.M. Zhang, Case-Based Reasoning in Design, Lawrence Erlbaum, 1995.
Fabel	Architecture	A. Voss, "Case Design Specialists in Fabel," in Issues and Applications of Case-Based Reasoning in Design, pp. 301–338.
Gencad	Structural systems	A. Gómez de Silva Garza and M.L. Maher, "The Adaptation of Structural System Designs Using Genetic Algorithms," Proc. Int'l Conf. Information Technology in Civil and Structural Engineering Design, Elsevier Science, New York, 1996, pp. 189–196.
Janus	Architecture	G. Fisher, R. McCall, and A. Morch, "Design Environments for Constructive and Argumentative Design," Proc. Human Factors in Computing Systems Conf., ACM Press, New York, 1989, pp. 269–275.
Julia	Meal planning	T.R. Hinrichs, "Towards an Architecture for Open World Problem Solving," Proc. CBR Workshop, Morgan Kaufmann, San Francisco, 1988, pp. 182–189.
Kritik, Kritik II	Physical devices	A. Goel, S. Bhatta, and E. Stroulia, "KRITIK: An Early Case-Based Design System," Issues and Applications of Case-Based Reasoning in Design, pp. 87–132.
Panda	Fire engines	C. Tsatsoulis and P. Alexander, "Integrating Cases, Sub-Cases, and Generic Prototypes for Design," in Issues and Applications of Case-Based Reasoning in Design, pp. 261–300.
Seed	Architecture	U. Flemming et al., "Case-Based Design in a Software Environment That Supports the Early Phases," in Issues and Applications of Case-Based Reasoning in Design, pp. 61–86.

意图。*Behavior variables* 是设计描述中与设计与其环境的交互及响应相关的特征。*Structure variables* 描述了设计的物理、几何和视觉属性。Gencad 引入了 *con-* 的概念，它用于描述可能影响设计决策的设计环境中的特征。

ariables

Fabel 通过采用不同的方法，应对建筑设计领域的复杂性。

特定设计的视图或方面。Fabel 系统可以将单一设计问题表示为多个设计案例，其中每个案例都是对该设计的不同视角。该方法采用设计的不同视图，例如：*gestalts* (复杂几何体的简化整体抽象)、*attribute vectors* (预定义特征的取值列表)以及*images* (设计方案的数字图像)。这一方法有别于基于部件的层级结构——

通过将每个视图视为一个案例，而不必将某个案例的所有视图关联起来以构成一个“完整”的案例描述。

多媒体方法通过将设计信息按不同存储格式进行组织，并在各表示形式之间建立交叉引用，从而应对复杂性。例如，在Archie-II中，一个设计可以由建筑的照片、其建筑平面图以及关于该建筑设计不同方面的文本注释组成。AskJef则为案例提供了符号、图形和听觉等多种表示形式。此外，AskJef还对案例进行时间上的分解，使得每个案例实际上是在设计阶段迭代生成的一系列不同版本的集合（其中仅最后一个版本被认定为优良方案）。

总之，设计人员通过将一个设计案例分解为更小、更简单的抽象层次来应对复杂性。表2总结了用于分解建筑设计案例的各类方法，并列举了相应的示例系统（示例系统的列表并不 exhaustive——本调查中提及的其他系统未在表2中列出，且许多未在此处提及的系统也可能采用上述每种方法）。

广义设计知识。将案例推理应用于涉及机械及其他物理装置的设计领域，已将设计案例的概念扩展为包含以因果或启发式知识形式表示的广义知识。在这些设计领域中，若缺乏一定的广义知识，设计案例的表示与重用便无法实现。按照案例推理设计系统的提出者所使用的术语，用于表示广义设计知识的一些方法包括

- 因果模型，
- 态相互作用，
- 启发式模型，
- 启发式规则，以及
- 几何约束。

Kritik 的开发者通过使用 *structure-behavior-function* 模型直接解决了因果行为的表示问题。（SBF 形式化与 FBS 形式化虽采用相同的术语，但其含义却截然不同。）Kritik 将行为表示为设备状态的集合以及连接这些状态的转换；这些行为解释了物理设备的结构元件如何

achieve its functions. Causal knowledge in the form of SBF models allows Kritik to diagnose and repair faulty or malfunctioning devices. Like Kritik, Cadet uses *interactive state diagrams* to represent device behavior, using graphs with nodes and transitions between the nodes. A device's behavior in Cadet, as in Kritik, explains how it achieves its functions.

In contrast to the models used in Kritik and Cadet, Casecad's FBS models identify a range of allowable values for each variable. These ranges define different models that cover sets of cases belonging to different classes, rather than a specific case, and are more like heuristic knowledge than causal knowledge.

Many systems represent generalized knowledge as constraints that define dependencies among variables. Cadsyn contains generalized knowledge in the form of constraints on the values of variables and their relationships that good designs must satisfy. By definition, these constraints apply to all design cases. Faming and Cadre store generalized knowledge as case-specific constraints.

Systems also use rules to capture generalized knowledge. Many implementations distinguish rules from constraints by applying rules to actively *change* a proposed design solution and constraints to evaluate the feasibility of a proposed design solution, corresponding to the modify and evaluate stages shown in Figure 2b. Panda uses rules to transform a retrieved design case to be feasible for a new design problem. Janus represents generalized knowledge in the form of rules that can help critique potential designs. Archie provides domain models as qualitative rules about how changes to one variable can affect other variables.

Table 3 summarizes the different approaches to representing generalized knowledge in case-based design systems. In some systems, the generalized knowledge dominates the representation of design cases; for example, Kritik's use of causal models provides the focus for case retrieval and adaptation. Other systems only need the generalized knowledge during case adaptation and represent it independently of a design case—for example, Panda's use of transformation rules.

Informal nature of domains. The lack of formal knowledge in design affects both the ability to define a formal and consistent representation of design cases and the role of adaptation as either a human-centered activity or an automated process. We can address the development of CBR for design domains having little formal or theoretical knowledge either by formalizing knowledge that previously was not formalized, or by identifying a representation of the design cases to support *human* rather than automated reasoning.

Cadre and Archie are CBR approaches to architectural design, a design domain characterized by a lack of formal, analytical knowledge. The Cadre project focuses on areas of architectural design that can be formalized, such as the geometric constraints on floor plans. Cadre represents design cases as existing floor plans and uses numerical constraints on the dimensions of the required floor plan to automatically modify and evaluate the adaptation and combination of floor plans.

The Archie project looks at how issues of importance during conceptual design can be articulated and presented to designers as multimedia. In this implementation, the design knowledge is not formalized for automated reasoning, but articulated and expressed in a form that aids people in reusing architectural principles. The use of multimedia and hypertext environments provides a rich representation and flexible use of the informal knowledge.

Julia develops a formal representation of a previously informal task, the task of meal planning, allowing it to be automated. Julia organizes the different types of meals as a taxonomy and articulates constraints on meal combination. Using a formalism of variables and their values, the meal-planning task involves recalling parts of previous meals followed by adapting and combining the cases using constraint satisfaction. In contrast to the Cadre system, Julia uses logical rather than numerical constraints.

Table 2. Approaches to complex design-case representation.

Approach	System	Example
Partonomic hierarchies	Casecad	Building as lateral and gravity systems.
Views	Fabel	Building as layout, topology, or gestalt.
Multimedia	Archie-II	Building as photographs, drawings, or stories.

Table 3. Approaches to representing generalized knowledge.

Approach	System	Example
Causal models	Kritik	Structure, behavior, and function interactions
State interactions	Cadet	Interactive state diagrams
Heuristic models	Casecad	Function, behavior, and structure attribute value ranges
Heuristic rules	Panda	Transformation rules
Geometric constraints	Cadre	Parametric constraints

Issues in developing case-based design systems

The application of CBR to design raises a common set of issues regardless of the application domain. These are the broad classifications of representation and process issues we discussed earlier. Representation issues include the content and representation of an individual design-case and case-memory organization. Process issues concern the general process model of a case-based design system—design-case recall and adaptation. Practitioners have addressed these issues in developing and implementing case-based design systems, where each system resolves the issues within the context of its project focus.

Representation of design cases and case memory. Before discussing the representation of design cases, let's look at how cases are considered in CBR. One view is that a case is a story or a lesson to be learned.[4] Other views express cases as information about the context as well as solutions of a problem,[7] or consider that cases record the process by which a problem is solved.[9] Whichever view you take, a case's purpose is to facilitate solving a similar problem in a similar but different context in the future.

A common issue raised in design applications is that a design cannot be told in a single case (or story), or from a single point of view. Design involves reasoning about a design artifact's rationale, structure, behavior, function, constraints among structural elements, decisions, and tradeoffs. A design case's size is an important consideration. Design cases can be divided into smaller chunks and their parts linked as a list, a hierarchy, or a network. The "Complex design cases" section gave examples of different decomposition schemes.

表2. 复杂设计案例的表示方法。

Approach	System	Example
Partonomic hierarchies	Casecad	Building as lateral and gravity systems.
Views	Fabel	Building as layout, topology, or gestalt.
Multimedia	Archie-II	Building as photographs, drawings, or stories.

表3. 表征广义知识的方法。

Approach	System	Example
Causal models	Kritik	Structure, behavior, and function interactions
State interactions	Cadet	Interactive state diagrams
Heuristic models	Casecad	Function, behavior, and structure attribute value ranges
Heuristic rules	Panda	Transformation rules
Geometric constraints	Cadre	Parametric constraints

实现其功能。以SBF模型形式呈现的因果知识使Kritik能够对存在故障或运行异常的设备进行诊断与修复。与Kritik类似，Cadet也采用*interactive*来表征设备行为，并使用由节点及节点间转移构成的图模型。在Cadet中，设备的行为与其在Kritik中的定义一致，均用于解释该设备如何实现其功能。

state diagrams

与Kritik和Cadet所采用的模型不同，Casecad的FBS模型为每个变量确定了一组允许取值范围。这些取值范围定义了不同的模型，这些模型覆盖了属于不同类别的一系列案例，而非针对某一特定案例，因此更接近于启发式知识，而非因果知识。

许多系统将广义知识表示为定义变量之间依赖关系的约束。Cadsyn以变量取值及其相互关系所应满足的约束形式存储广义知识；根据定义，这些约束适用于所有设计场景。Faming和Cadre则以特定于具体案例的约束形式存储广义知识。

系统还利用规则来捕捉概括性知识。许多实现方案通过以下方式区分规则与约束：规则用于主动地对所提出的设计方案进行修改，而约束则用于评估该方案的可行性，这分别对应于图2b所示的“修改”和“评估”两个阶段。Panda利用规则将检索到的设计案例转换为适用于新设计问题的可行方案。Janus以规则的形式表示概括性知识，这些规则有助于对潜在设计方案进行批判性评价。Archie则以定性规则的形式提供领域模型，描述某一变量的变化如何影响其他变量。

表3总结了在基于案例的设计系统中表示广义知识的不同方法。在某些系统中，广义知识主导着设计案例的表示；例如，Kritik通过使用因果模型来确定案例检索与适应的重点。而在另一些系统中，广义知识仅在案例适应过程中发挥作用，并且以独立于设计案例的形式进行表示——例如，Panda采用变换规则的方式。

领域的非正式性。设计中缺乏形式化知识，这既影响了定义形式化且一致的表示的能力，设计案例的呈现以及适应性在人本活动与自动化过程中的角色。对于那些缺乏充分形式化或理论基础的设计领域，我们可以通过对先前未形式化的知识进行形式化，或者通过确定一种能够支持*human*而非自动化推理的设计案例表示方法，来推进基于案例的推理技术的发展。

Cadre和Archie是面向建筑设计的案例推理方法，而建筑设计领域的一个显著特征是缺乏形式化的分析性知识。Cadre项目主要关注那些可以被形式化的设计方面，例如平面布局的几何约束。Cadre将设计实例表示为现有的平面布局，并利用对所需平面尺寸的数值约束，自动调整并评估平面布局的适应与组合过程。

Archie项目旨在研究如何将概念设计阶段的重要问题以多媒体形式向设计师进行表达与呈现。在该实现中，设计知识并未被形式化以支持自动化推理，而是以一种有助于人们复用建筑原则的形式加以阐述与表达。多媒体与超文本环境的运用为非正式知识提供了丰富的表征方式和灵活的使用途径。

Julia为一项此前非正式的任务——膳食计划——构建了一种形式化表示，从而使其能够实现自动化。Julia将不同类型的膳食组织成一个分类体系，并明确了膳食组合的约束条件。借助变量及其取值的形式化框架，该膳食计划任务包括回顾先前膳食的部分内容，随后通过约束满足方法对这些案例进行调整与组合。与Cadre系统不同，Julia采用的是逻辑约束而非数值约束。

基于案例的设计系统开发中的问题

将案例推理技术应用于设计时，无论具体应用领域如何，都会引发一系列共性问题。这些问题可归结为我们在前文中讨论的两大类：表示问题与过程问题。表示问题涉及单个设计案例的内容及其表示方式，以及案例库的组织结构；过程问题则关乎基于案例的设计系统的一般过程模型——即设计案例的检索与适应。在开发和实施基于案例的设计系统的过程中，从业者已针对这些问题提出了相应的解决方案，而每个系统均在其特定项目背景下对这些问题加以具体化解。

设计案例的表示与案例记忆。在讨论设计案例的表示之前，我们先考察案例在基于案例的推理（CBR）中是如何被看待的。一种观点认为，案例是一个有待汲取的经验或教训；[4] 另一些观点则将案例视为关于问题情境及其解决方案的信息；[7] 还有观点认为，案例记录了问题求解的过程。[9] 无论持何种观点，案例的根本目的在于为未来在相似但有所不同的情境中解决类似问题提供便利。

在设计应用中经常遇到的一个问题是，一个设计无法通过单一案例（或故事）或从单一视角来完整呈现。设计涉及对设计对象的 rationale、结构、行为、功能、各结构元素之间的约束关系、决策以及权衡取舍等进行推理。设计案例的规模是一个重要的考量因素。设计案例可以被分解为更小的单元，并将其各部分以列表、层级结构或网络的形式加以关联。“复杂设计案例”一节给出了不同分解方案的示例。

Table 4. What is in a design case?

Approach	System
Design lessons	Archie, Archie-II, AskJef
Function, behavior, and structure variables	Casecad, Cadsyn, Gencad
Formal causal models that explain	Kritik, Cadet
Interpretations as drawing, function, and behavior models	Cadre, Faming
Plan actions and components of physical systems	Panda
Specification units, method to create the solution	Seed
Gestalts, images, vectors, and tuples	Fabel

Table 5. Design-case retrieval methods.

Method	System
User-directed browsing and navigation	Archie, Archie-II, Casecad, AskJef, Janus
Attribute-value matching	Cadsyn, Casecad, Seed, Julia, Fabel, Asp-II, Benton, Panda
Functional indexing	Kritik
Image- and gestalt-based pattern matching	Fabel
Hierarchical problem specifications matching	Seed
Graph matching	Cadet

Case-memory organization refers to the way cases are organized for access during retrieval. Where case memory does not contain many cases, the memory organization is usually a list of pointers to cases. A large case memory requires more efficient access to cases or subcases. AI practitioners commonly use induction to generate a hierarchical indexing tree whose leaf nodes are associated with a list of pointers to cases or subcases. A common issue in organizing design cases is the need to predetermine the features to serve as indices for efficient access to cases. It is difficult, however, to decide beforehand the subset of features that are likely to help in identifying relevance to a new design problem.

Designers resolve the issue of "What is in a design case?" in many ways. Most design stories told by designers or found in design documents do not easily form into cases that can be indexed and classified for reuse. A systematic approach is needed to identify uniform representations and to parse design stories into these formats. The variety of ways for representing designs provides a richer presentation of design information to the user and can directly influence retrieval and adaptation methods.

Design cases can be lessons for helping designers interpret a previous design experience in a similar context, as do Archie, Archie-II, and AskJef. These systems emphasize—both from the point of view of internal representation and in the way they present cases to the user—the results, both good and bad, of key design decisions.

Design cases can classify their variables according to which ones relate to their function, behavior, and structure, so that a case's retrieval and adaptation can focus on one or more of these aspects of a design, as in Casecad and Gencad. These systems emphasize the need for processing different subsets of a design's variables according to the different interests that a user might have at different stages in the design process.

Design cases can consist of models that explain how their structure achieves their functions, as in Kritik and Cadet. In these systems, a case's structure is the solution to a design problem, its function is the design problem that the case solved, and the behavior is an explanation of how the structure achieves its functions. These systems represent and use each of these three types of information differently. In contrast, Casecad and Gencad uniformly represent function, behavior, and structure as attribute-value pairs. Systems using causal models emphasize the ability to use the knowledge embodied in the models to understand a design's internal functioning, and thus to automatically adapt the case.

Design cases can consist of *interpretations*, or a collection of different types of models, as in Cadre and Faming. In these systems, a case links to an interpretation, which consists of three (interrelated) models: a CAD model of the structure of the design, a behavioral model of what the structure does, and a functional model of what is important about what the structure does. As in Kritik and Cadet, these systems emphasize the representation of causal knowledge about designs, but unlike Kritik and Cadet, the representation is done through several interrelated models, represented separately, rather than through one model consisting of several types of information represented together.

Design cases can contain plan actions, components of physical systems, and specification elements, as in Panda and Seed. These systems use the different components of a case representation for different reasoning tasks. For example, they use the specification part of the case during retrieval and the plans during case adaptation.

Another approach represents a case as different views of the same design. For example, in Fabel, design cases can contain images, gestalts, and tuples of values. This project shows that there is no simple response to "What is in a design case?" but rather many answers. The Fabel project also shows that there can be many answers within a single implementation. Representing each case in a variety of ways extends its usefulness to many different situations.

We can consider the representation of design cases according to the way they are stored as computer files, such as text, CAD drawings, and object-oriented models. Many case-based design systems combine representation techniques, as in Cadre, Casecad, Archie, Archie-II, and AskJef (which also has auditory information about cases). These systems emphasize facilitating the presentation of information about a case to the user (that is, information in one or more user-friendly formats), while still permitting the system to operate on the cases.

Design cases can contain a solution generated by a design module, an associated problem specification, and the technology or method used to create the solution, as in Seed. This type of system emphasizes the ability to adapt the procedures used to solve previous problems in new, but similar, contexts, as opposed to only adapting the solution itself.

In summary, the most important issue when defining a case representation is what information in the design facilitates its reuse. Then, the issue of how the CBR tool uses the design-case representation as part of the design process (for retrieval only, for adaptation only, or for both retrieval and adaptation) helps determine the implemented representation of a design case. Table 4 summarizes the approaches of the systems surveyed for this article, using the terminology of the authors of the systems. The approaches are as numerous as there are systems, indicating that this aspect of case-based design has not matured enough to lead to general principles of how to represent a design case.

Table 4. What is in a design case?

Approach	System
Design lessons	Archie, Archie-II, AskJef
Function, behavior, and structure variables	Casecad, Cadsyn, Gencad
Formal causal models that explain	Kritik, Cadet
Interpretations as drawing, function, and behavior models	Cadre, Faming
Plan actions and components of physical systems	Panda
Specification units, method to create the solution	Seed
Gestalts, images, vectors, and tuples	Fabel

Table 5. Design-case retrieval methods.

Method	System
User-directed browsing and navigation	Archie, Archie-II, Casecad, AskJef, Janus
Attribute-value matching	Cadsyn, Casecad, Seed, Julia, Fabel, Asp-II, Benton, Panda
Functional indexing	Kritik
Image- and gestalt-based pattern matching	Fabel
Hierarchical problem specifications matching	Seed
Graph matching	Cadet

案例记忆组织是指在检索过程中用于访问案例的组织方式。当案例记忆中包含的案例数量较少时，其组织形式通常为指向各案例的指针列表。而当案例记忆规模较大时，则需要更为高效的案例或子案例访问机制。人工智能领域的从业者常采用归纳法构建层次化索引树，其叶节点与指向案例或子案例的指针列表相关联。在设计案例的组织过程中，一个常见的问题在于必须预先确定用作索引的特征，以实现对案例的高效访问。然而，事先难以判定哪些特征子集能够有效帮助识别与新设计问题的相关性。

设计者们以多种方式解决“设计案例中包含哪些内容”这一问题。大多数由设计者讲述或存在于设计文档中的设计故事，难以直接转化为可供索引和分类以供复用的案例。因此，需要采用系统化的方法来识别统一的表达形式，并将设计故事解析为这些标准化格式。设计表示形式的多样性能够为用户提供更为丰富的设计信息呈现，并可能直接影响检索与适应方法的选择。

设计案例可以为设计师在类似情境下解读过往设计经验提供借鉴，正如Archie、Archie-II和AskJef所示。这些系统无论从内部表征的角度，还是在向用户呈现案例的方式上，均强调关键设计决策所带来的正面与负面结果。

设计案例可以根据其变量与功能的相关性对其进行分类。特性、行为和结构，从而使案例的检索与适配能够聚焦于设计的某一个或多个方面，正如Casecad和Gencad系统所体现的那样。这些系统强调，在设计过程的不同阶段，应根据用户可能存在的不同关注点，分别处理设计变量的不同子集。

设计案例可以由解释其结构如何实现功能的模型构成，如Kritik和Cadet系统所示。在这些系统中，案例的结构即为某一设计问题的解决方案，其功能则是该案例所解决的设计问题，而行为则用于说明该结构如何实现其功能。这些系统对上述三类信息的表示与使用方式各不相同。相比之下，Casecad和Gencad则将功能、行为和结构统一表示为属性—值对。采用因果模型的系统则强调利用模型中蕴含的知识来理解设计的内部运行机制，从而实现案例的自动适应。

设计案例可以由*interpretations*构成，也可以是由多种不同类型模型组成的集合，如Cadre和Faming系统所示。在这些系统中，一个案例与一种解释相关联，而该解释由三个相互关联的模型构成：设计结构的CAD模型、描述该结构功能的行为模型，以及刻画该结构功能重要性的功能模型。与Kritik和Cadet系统类似，这些系统同样强调对设计因果知识的表示，但与Kritik和Cadet不同的是，这种表示是通过若干相互关联且分别独立建模的方式来实现的，而非通过单一模型来完成。由多种信息类型共同表示的模型。

设计案例可以包含计划行动、物理系统的组成部分以及规范元素，正如Panda和Seed系统中所采用的那样。这些系统针对不同的推理任务，分别使用案例表示中的不同组件。例如，在案例检索过程中使用规范部分，而在案例适配过程中则使用计划部分。

另一种方法将每个案例视为同一设计的不同视图。例如，在Fabel中，设计案例可以包含图像、整体构型以及值的元组。该项目表明，“设计案例中包含什么”这一问题并不存在单一的答案，而是存在多种解答。此外，Fabel项目还表明，在单一实现中也可能存在多种不同的答案。以多种方式表示每个案例，能够将其适用性扩展至多种不同的情境。

我们可以根据设计案例在计算机文件中的存储方式来考虑其表示形式，例如文本、CAD图纸以及面向对象的模型。许多基于案例的设计系统会综合运用多种表示技术，如Cadre、Casecad、Archie、Archie-II以及AskJef（该系统还包含关于案例的听觉信息）。这些系统侧重于便于向用户呈现案例相关信息（即以一种或多种用户友好的格式展示信息），同时仍允许系统对这些案例进行处理。

设计案例可包含由设计模块生成的解决方案、相关的问题规范，以及用于构建该解决方案的技术或方法，如Seed系统所示。此类系统强调在新的但相似的情境中对解决先前问题所采用的程序进行适应性调整，而不仅限于对解决方案本身的调整。

总之，在定义案例表示时，最重要的问题在于设计中的哪些信息有助于其重用。此外，CBR工具在设计过程中如何利用设计案例表示（仅用于检索、仅用于适应，或同时用于检索与适应）这一问题，也有助于确定所采用的设计案例表示形式。表4总结了本文所调研的各系统采用的方法，并沿用了各系统作者的术语。由于方法的数量与系统的数量相当，这表明基于案例的设计在这一方面尚未成熟，尚不足以形成关于如何表示设计案例的一般性原则。

Indexing and retrieving design cases. In CBR, recalling a case assumes that you know what you are looking for—that is, that the problem is well-defined. Design often involves solving an ill-defined problem in which part of the design process is defining the problem in addition to developing a solution. Designers thus cannot perform the indexing and retrieval involved in design-case recall simply as direct pattern matching. In many implementations, the indexing and retrieval of design cases is done informally, where the user browses through the case library and selects a relevant design case.

The use of informal approaches to recalling relevant design cases addresses the need for flexibility in developing an understanding of the new design problem through browsing and retrieving design cases. The informal approach's effectiveness depends on the number of designs in case memory and the richness of the indexing scheme. If the case library is very large, the informal approach is not very efficient. Linking and cross-indexing the cases allows the user to navigate and browse through the cases in the case library that are related in different ways.

Informal approaches are usually implemented as hypertext browsers with some capability for word search. These systems usually only provide for case browsing and do not help with design-case adaptation. In some systems, such as Cadre, the number of design cases is so small that the user can browse through all stored cases to find the one best for adaptation, and the system assists with the adaptation.

Formal approaches to recalling design cases make assumptions about how a new problem is specified and use the specification for pattern matching. The most common formal method for design-case recall uses a set of attribute-value pairs to specify a new design problem, then matches this set with design cases in memory. A close match is a design case that has many attribute-value pairs in common with the specifications. Many implementations use a *weighted-sum* approach to measure the closeness of the match, where weights define the relative importance of a particular attribute. Although various algorithms are used for pattern matching, many systems use the general approach of matching attribute-value pairs. These include Casecad, Cadsyn, Julia, Fabel, Seed, Asp-II, Panda, and Benton.

Other approaches to finding a relevant case are to index and retrieve based on function, as in Kritik; to retrieve based on matching images or gestalts, as in Fabel; to retrieve based on a hierarchy of problem specifications, as in Seed; and to retrieve using graph-based representations of behavior, as in Cadet. Function retrieval, as in Kritik, uses a single function as the index to each case. This approach describes the new design problem by its intended function and retrieves cases that provide that function. Fabel uses richer pattern-matching methods than the weighted-sum approach; for example, the *fish and sink* algorithm measures the distance between a case and a new problem description. Table 5 summarizes the approaches to case retrieval taken in the systems included for this survey.

A common issue in developing indexing and retrieval schemes for design cases is the need for flexibility and for allowing the design specifications to change or be refined. For example, Kritik analyzes a memory of substances, part of its general domain knowledge, to elaborate on the original design-problem specifications. Casecad and Cadet provide strategies for modifying the new design specification based on the cases retrieved.

APPROACHES ARE AS NUMEROUS AS THERE ARE SYSTEMS, INDICATING THAT THIS ASPECT OF CASE-BASED DESIGN HAS NOT MATURED ENOUGH TO LEAD TO GENERAL PRINCIPLES OF HOW TO REPRESENT A DESIGN CASE.

Another retrieval-related issue is the level of abstraction of cases and the need to decompose a problem to find a relevant design case or subcase. For example, Julia and Cadsyn decompose the problem specification if they cannot find a case to solve the entire problem. The retrieval process then considers a subset of the entire design problem when looking for relevant design cases.

Design-case adaptation. The process of design-case adaptation is essentially the synthesis of a new design solution. The recall of one or more similar design cases provides the starting point for generating a new solution. The adaptation of design cases is the part of case-based design where a new design is generated. CBR applications to design fall into three categories, depending on who or what performs the adaptation:

- the *human designer*,
- a *knowledge-intensive computational method*, or
- a *knowledge-lean computational method*.

Human design-case adaptation. Some systems described here are intended to serve as case libraries—repositories of information about specific designs represented using different formats, with the cases cross-linked to facilitate navigation among related items in the case library. They usually emphasize facilitating case-memory browsing and navigation, and case retrieval. These systems leave what is done with the cases encountered in the memory to the user. Systems taking this approach include Archie, Archie-II, AskJef, Casecad, and Janus. Although these systems do not tackle case adaptation, Archie and Janus do include support for evaluating the user's designs (presumably adapted by the user from previous cases) on the basis of domain models and rules.

Knowledge-intensive case adaptation methods. In many of the applications of CBR to design, automating design-case adaptation relies on constraint satisfaction—for example, in Julia, Cadsyn, Cadre, and Faming (although in Julia it is called constraint posting). Julia handles dynamic constraint variables as well; that is, as the adaptation of a case proceeds, Julia can discover new constraints (or users can specify new ones) that apply to the problem, or it can drop some of the previously considered constraints. Cadsyn uses a predefined set of constraints to evaluate proposed design solutions.

Faming uses numerical constraints to adapt the geometry of a proposed floor plan that is generated by combining cases. Called *dimensional adaptation*, the method consists of dimensionality expansion and reduction. The adaptation strategies in both Faming and Cadre represent the problems they have to solve as networks of constraints; thus, these systems implement case adaptation using algorithms that solve constraint-satisfaction problems. The final solutions provided by constraint-satisfaction algorithms, if any, will not need evaluation because they will be guaranteed to satisfy the constraints.

设计案例的索引与检索。在基于案例的推理中，案例的召回假定用户已明确所寻求的目标，即问题已被清晰界定。然而，设计过程往往涉及解决一类定义不甚明确的问题，其中设计过程的一部分不仅在于提出解决方案，还在于对问题本身的界定。因此，设计师无法仅通过直接的模式匹配来完成设计案例召回中的索引与检索操作。在许多具体实现中，设计案例的索引与检索是以非正式的方式进行的，即由用户浏览案例库并从中选择一个相关的设计案例。

采用非正式方法来检索相关设计案例，旨在满足在浏览与检索设计案例过程中灵活构建对新设计问题理解的需求。该非正式方法的有效性取决于案例库中设计实例的数量以及索引方案的丰富程度。若案例库规模庞大，该方法的效率则相对较低。通过建立案例间的关联并进行交叉索引，用户能够以多种关联方式在案例库中进行导航与浏览。

非正式方法通常以超文本浏览器的形式实现，并具备一定的关键词检索功能。这些系统往往仅支持案例浏览，而无法辅助设计案例的适配。在某些系统中，例如Cadre，设计案例的数量非常有限，用户可以通过遍历所有已存储的案例来找到最适合进行适配的那个，且系统会为适配过程提供相应的支持。

用于设计案例检索的正式方法会对新问题的描述方式作出假设，并利用该描述进行模式匹配。最常见的设计案例检索形式化方法是采用一组属性-值对来描述新的设计问题，然后将该集合与存储中的设计案例进行匹配。当一个设计案例与问题描述具有大量共同的属性-值对时，即被视为高度匹配。许多实现采用v0方法来度量匹配的紧密程度，其中权重用于确定某一属性的相对重要性。尽管在模式匹配中使用了多种算法，但许多系统仍沿用基于属性-值对匹配的通用方法。这些系统包括Casecad、Cadsyn、Julia、Fabel、Seed、Asp-II、Panda和Benton。

寻找相关案例的其他方法是基于功能进行索引与检索。如Kritik所示，通过匹配图像或格式塔特征进行检索；如Fabel所示，基于问题规格的层次结构进行检索；如Seed所示，通过基于图的表征来检索行为模式；以及如Cadet所示，利用基于图的表征进行检索。在Kritik中，函数检索以单一函数作为每个案例的索引。该方法通过设计目标函数来描述新的设计问题，并检索能够实现该功能的案例。Fabel采用比加权求和法更为丰富的模式匹配方法；例如，*fish and sink*算法用于度量案例与新问题描述之间的距离。表5总结了本综述所涵盖的各系统在案例检索方面所采用的方法。

在索引开发中，一个常见问题是

APPROACHES ARE AS NUMEROUS AS THERE ARE SYSTEMS, INDICATING THAT THIS ASPECT OF CASE-BASED DESIGN HAS NOT MATURED ENOUGH TO LEAD TO GENERAL PRINCIPLES OF HOW TO REPRESENT A DESIGN CASE.

设计案例的存储与检索方案需要具备灵活性，以适应设计规范的变更或细化。例如，Kritik 通过分析其通用领域知识中关于物质的记忆，来细化初始的设计问题规范。Casecad 和 Cadet 则提供了基于检索到的案例对新设计规范进行修改的策略。

另一个与检索相关的问题是案例的抽象层次，以及为找到相关的设计案例或子案例而对问题进行分解的必要性。例如，当无法找到能够解决整个问题的案例时，Julia 和 Cadsyn 会将问题规范进行分解；随后，在检索过程中，系统会在寻找相关设计案例时仅考虑整个设计问题的一个子集。

设计案例适配。设计案例适配过程本质上是对新设计方案的综合生成。通过调用一个或多个相似的设计案例，为生成新方案提供初始依据。

设计案例的适配是基于案例的设计过程中生成新设计方案的环节。面向设计的案例推理应用根据执行适配的主体不同，可分为三类：

- *human designer*，
- a *knowledge-intensive computational*，或 *method*
- a *knowledge-lean computational method*。

Human design-case adaptation. 本文所述的某些系统旨在作为案例库——即以不同格式表示的特定设计信息的存储库，并通过案例间的交叉链接来促进相关条目之间的导航。这些系统通常侧重于支持案例记忆的浏览与导航，以及案例检索。对于在案例记忆中遇到的案例如何处理，则交由用户自行决定。采用这种思路的系统包括 Archie、Archie-II、Ask Jef、Casecad 和 Janus。尽管这些系统并未直接解决案例适配问题，但 Archie 和 Janus 确实提供了基于领域模型和规则对用户设计（ presumably 由用户从前例中适配而来）进行评估的支持。

Knowledge-intensive case adaptation methods. 在许多将案例推理应用于设计的场景中，设计案例的自适应过程依赖于约束满足——例如，在Julia、Cadsyn、Cadre和Faming等系统中（尽管在Julia中这一过程被称为约束后置）。Julia还能够处理动态约束变量；也就是说，随着案例自适应的推进，Julia可以发现新的约束（或由用户指定新的约束），这些约束适用于当前问题，或者也可以删除部分先前已考虑的约束。Cadsyn则使用一组预定义的约束来评估所提出的设计方案。

Faming 采用数值约束来调整由案例组合生成的拟议平面布局的几何形态。该方法被称为 *dimensional adaptation*,，其核心在于维度的扩展与约简。在 Faming 和 Cadre 中，适应策略均将待求解的问题表述为约束网络；因此，这些系统通过求解约束满足问题的算法来实现案例的适应。若约束满足算法能够给出最终解，则无需再进行评估，因为这些解必然满足所有约束条件。

Table 6. Methods for automated design-case adaptation.

METHOD	SYSTEM
Constraint satisfaction	Cadsyn, Cadre, Faming, Fabel, Julia
Rule-based transformation	Panda, Asp-II, Benton
Subcase combination	Seed, Cadsyn, Panda
Simulation	Asp-II, Kritik, Benton
Causal diagnose and repair	Kritik
Design plan replay	Seed, Asp-II, Benton
Genetic algorithm	Gencad
Generate and test	Fabel

Instead of constraints, other systems employ heuristic modification rules (some of which can implicitly embody constraints) that are specific to the domain, but valid for all artifacts in the domain. Panda, Asp-II, and Benton use a rule-based approach to tweak the parameters in a retrieved case to transform it into a solution to the new problem.

Panda attempts a technique called *reinstantiation* or *subcase replacement* as an alternative for case adaptation. In this method, a submodule from one case adds to, or replaces a submodule in, another case. Panda evaluates a design by comparing the goals it satisfies with the initial user-specified goals. Asp-II uses two alternative, but similar, methods. It uses replanning to adapt a submodule of a case when a rule-based method has successfully adapted the rest of the case. It uses instantiation to adapt a generic description of a design solution by finding appropriate values for its variables when no specific cases that can be adapted have been retrieved from memory. Asp-II evaluates designs both by performing a simulation and by searching for similar cases in memory that represent *failed* designs. Benton employs Asp-II's replanning method for case adaptation to replace fragments of both the design solutions and the design processes stored in its cases. Benton uses syntactic and other knowledge related to its domain to verify the correctness of its designs.

Another case-adaptation approach combines model-based reasoning with CBR. The cases provide the starting point of the solution to a given problem. Models provide knowledge to fix the areas where cases do not quite satisfy the current problem specification. Kritik considers case adaptation to be a diagnosis and repair problem, and implements case evaluation as a simulation process. Case-specific models include behaviors, which explain how a design's structure achieves its function. Kritik analyzes (diagnoses) these models and identifies candidate modifications (repairs). Case adaptation is plan-based—the system knows and can attempt different methods or plans for adaptation. Generic adaptation methods in Kritik include component replacement, component replication, and substance replacement. After performing a repair, Kritik simulates the resulting model to determine whether the resulting structure achieves the intended functions, and thus whether adaptation worked.

Fabel offers a variety of adaptation methods that depend on the aspect or view of the case being adapted, and uses techniques such as generate-and-test and constraint satisfaction. Thus, as in many of the other systems we've discussed, Fabel adopts a hybrid framework combining several methods.

An alternative view of case adaptation lets the user interact with the system, thus combining the automation of tedious, repetitive, formalizable tasks, with the intelligent decision-making capabilities of users in important stages of the design. For example, Seed can autonomously perform several case-adaptation operations, but the user must select which ones to attempt at which times. The case-adaptation operations available to the user in Seed implement the addition, removal, translation, modification, aggregation, and disaggregation of design components, and the alteration of their attributes. Seed supports the evaluation of designs with respect to user-specified design requirements.

Knowledge-lean case adaptation. Case adaptation and evaluation in the systems we've described rely on access to generic domain knowledge such as constraints, rules, plans, or models. However, systems can also perform case adaptation without relying on such domain knowledge. Our current work on Gencad focuses on design-case adaptation. Within the subtasks of case adaptation, we employ a genetic algorithm to eliminate the need for expertise related to both the proposal and the modification of potential solutions to design problems.

Gencad contains a case library of 61 structural engineering designs of tall buildings. It retrieves all partially matching cases when a new design problem is specified. The retrieved cases serve as the initial population of Gencad's genetic algorithm. The genetic algorithm uses both crossover and mutation to generate variations on the population's cases, each of which is a potential solution to the design problem. Domain knowledge in the form of constraints, together with the user's design-problem specifications, assist the evaluation of potential solutions. If none of them is good enough, Gencad selects the best of the potential solutions generated and left over from previous generations of the genetic algorithm to participate in a new round of crossover, mutation, and evaluation, and the process repeats.

In further research on Gencad, we're trying to eliminate the need for large amounts of knowledge for evaluating potential design solutions, as well as for their generation and modification. We will do this by training a neural network to recognize good solutions (using the 61 cases in Gencad's memory as examples of good solutions) and using it to evaluate the solutions resulting from the genetic algorithm. The neural network will eliminate the need to explicitly code either constraints or heuristic rules about the domain of structural engineering that are currently necessary to evaluate the results produced by the genetic algorithm.

Summary of automated design-case adaptation methods. The choice of case adaptation methods used by the different systems depends largely on what kind of knowledge is available about a domain, as well as the form in which the knowledge, and the design cases themselves, is available or represented. A general model for design-case adaptation is to propose a design solution, evaluate the proposed design solution, and modify the design until evaluation is satisfactory. However, this general description does not provide any insight into how to implement design-case adaptation. Table 6 summarizes the systems described just now.

THE USE OF CBR IN DESIGN HAS grown as a rich source of inspiration for how to apply the CBR paradigm to a new design application. The variety and complexity of design domains that have demonstrated the use of CBR have addressed many issues that the paradigm itself does not directly address. The most consuming effort in developing a case-based design system is the development of the case memory. This is equivalent to developing the knowledge base or grammar for the knowledge-based and language approaches to modeling design.

On the other hand, the development of

表6. 自动化设计案例适配方法。

METHOD	SYSTEM
Constraint satisfaction	Cadsyn, Cadre, Faming, Fabel, Julia
Rule-based transformation	Panda, Asp-II, Benton
Subcase combination	Seed, Cadsyn, Panda
Simulation	Asp-II, Kritik, Benton
Causal diagnose and repair	Kritik
Design plan replay	Seed, Asp-II, Benton
Genetic algorithm	Gencad
Generate and test	Fabel

与其他系统不同，这些系统并未采用约束条件，而是使用特定于该领域的启发式修改规则（其中部分规则可隐式地体现约束），且这些规则对领域中的所有实例均有效。Panda、Asp-II 和 Benton 采用基于规则的方法，通过调整检索到的案例中的参数，将其转化为新问题的解决方案。

Panda 尝试采用一种称为 *rein-* 或 *stantiation* *subcase replacement* 的技术作为案例适配的替代方案。在该方法中，某一案例中的子模块会添加到另一案例中，或替换其子模块。Panda 通过比较设计所满足的目标与用户最初指定的目标来评估设计方案。Asp-II 则采用两种虽有差异但相似的方法：当基于规则的方法已成功适配案例的其余部分时，它利用重规划来适配案例中的某个子模块；而在内存中未检索到可直接适配的具体案例时，则通过实例化为设计解决方案的通用描述赋予其变量适当的取值。Asp-II 既通过仿真运行来评估设计方案，又在内存中搜索与之相似的、代表 *failed* 设计的案例。Benton 借鉴 Asp-II 的重规划方法进行案例适配，以替换其案例中存储的设计方案和设计过程的片段。同时，Benton 还利用与其领域相关的句法及其他知识来验证其设计的正确性。

另一种案例自适应方法将基于模型的推理与案例推理相结合。案例为给定问题的求解提供了初始起点，而模型则提供知识，用于修正那些无法完全满足当前问题规范的环节。Kritik 将案例自适应视为一个诊断与修复问题，并将案例评估实现为一个仿真过程。特定于案例的模型中包含行为描述，用以说明设计结构如何实现其功能。Kritik 对这些模型进行分析（诊断），并识别出候选的修改方案（修复）。案例自适应是以规划为基础的——系统不仅知晓多种自适应方法或规划，而且能够尝试实施这些方法或规划。通用的自适应方法在Kritik 包括部件更换、部件复制以及物质替换。在完成一次修复后，Kritik 会对所得模型进行仿真，以判定所得结构是否实现了预期功能。

因此，适应是否有效。

Fabel 提供了多种适应方法，这些方法取决于所适应案例的特定方面或视图，并采用诸如生成与测试以及约束满足等技术。因此，正如我们讨论过的许多其他系统一样，Fabel 也采用了融合多种方法的混合框架。

另一种关于案例适配的视角允许用户与系统进行交互，从而将繁琐、重复且可形式化任务的自动化处理与用户在设计关键阶段的智能决策能力相结合。例如，Seed 能够自主执行多项案例适配操作，但用户必须决定在何时尝试哪些操作。Seed 中可供用户使用的案例适配操作包括设计组件的添加、删除、平移、修改、聚合与拆分，以及对其属性的变更。此外，Seed 还支持根据用户指定的设计需求对设计方案进行评估。

Knowledge-lean case adaptation. 在我们所描述的系统中，案例适配与评估依赖于对诸如约束、规则、计划或模型等通用领域知识的访问。然而，系统也可以在不依赖此类领域知识的情况下进行案例适配。我们当前在Gencad项目中的研究重点是设计案例的适配。在案例适配的子任务中，我们采用遗传算法来消除对设计问题潜在解决方案的提出与修改所需的专业知识依赖。

Gencad 包含一个由61个高层建筑结构设计方案组成的案例库。当指定一个新的设计问题时，它会检索出所有部分匹配的案例。这些检索到的案例作为 Gencad 遗传算法的初始种群。该遗传算法同时采用交叉和变异操作，对种群中的各个案例进行变异，从而生成新的变体，每一个变体都可视为该设计问题的一个潜在解决方案。领域知识以约束的形式与之相结合，连同根据用户的设计问题规范，辅助对潜在方案进行评估。若所有方案均不理想，Gencad将从先前各代遗传算法中生成并遗留的最佳潜在方案中择优，使其参与新一轮的交叉、变异与评估，随后该过程循环往复。

在Gencad的进一步研究中，我们正致力于消除对大量知识的依赖，这些知识原本用于评估潜在的设计方案以及对其进行生成和修改。为此，我们将训练一个神经网络以识别优良方案（以Gencad记忆库中的61个案例作为优良方案的示例），并利用该网络对遗传算法所生成的方案进行评估。通过这一方法，神经网络将使我们无需再显式地编码当前用于评估遗传算法输出结果的结构工程领域约束条件或启发式规则。

Summary of automated design-case adaptation methods. 不同系统所采用的案例适配方法在很大程度上取决于关于该领域的知识类型，以及这些知识和设计案例本身的存在形式或表示方式。设计案例适配的一般模型是：首先提出一个设计方案，然后对该方案进行评估，并在评估结果不令人满意时对方案进行修改，直至达到满意的评估标准。然而，这一概括性的描述并未提供有关如何具体实施设计案例适配的任何具体指导。表6总结了上述各系统的特点。

在设计领域中，基于案例的推理（CBR）的应用日益广泛，为如何将CBR范式应用于新的设计任务提供了丰富的启发。已成功运用CBR的设计领域之多样性和复杂性，也揭示了该范式本身并未直接涉及的诸多问题。在构建基于案例的设计系统时，最为耗时的工作在于案例库的构建，这相当于为基于知识的方法和基于语言的方法在设计建模中的应用开发知识库或语法规则。

另一方面，……的发展

Coming in June

QUALITATIVE REASONING

Qualitative understanding of behavior and the capability to reason with qualitative knowledge remains crucial in both human and machine intelligence despite—or some would say because—more and more sophisticated techniques are used in the modeling and analysis of everything from electro-mechanical devices to such nonphysical systems as the stock market. Guest edited by Yumi Iwasaki of Stanford's Knowledge Systems Lab, this special issue will document recent progress in this area by looking at

- **Modeling and simulation of complex industrial processes**
- **Model-based automatic generation of sequence control programs from design information**
- **Gas turbine condition monitoring using qualitative model-based diagnosis**
- **Using qualitative physics to create articulate software for science and engineering education**
- **Qualitative simulation and prediction**
- **Qualitative models as indices for memory-based prediction**
- **NASA's current work on autonomous spacecraft**

More AI News You Can Use

case-based design systems has inspired much research on AI in design. The use of CBR as an umbrella for understanding how to apply formal models of design knowledge or experience to a new design problem has become common. One example of this hybrid approach is the combination of case- and model-based reasoning. The concepts in CBR for memory organization, indexing, and retrieval can apply directly to model-based reasoning. We see the distinction blur in systems such as Kritik. The generality of the CBR paradigm shows that it can be applied to many AI approaches to design.

Finally, the issue of design assistance or design automation has led to two types of case-based design systems: those that provide a design memory with facilities for indexing and retrieving previous designs, and those that automatically retrieve or adapt previous designs. The distinction refers to different expectations about the role of the user, or human designer. This distinction is not a major issue in our presentation of case-based design systems. The issues raised by applying CBR to design center around the representation of design cases. The development of a design-case memory is informed by AI techniques and design research. The use of the design memory for design support or design automation might influence the user-interface design and the development of algorithms for changing design cases, but does not change the need to develop formal representations of design experiences.

References

1. R.C. Schank, *Dynamic Memory: A Theory of Learning in Computers and People*, Cambridge Univ. Press, Cambridge, UK, 1982.
2. J.L. Kolodner and C.K. Riesbeck, *Experience, Memory, and Reasoning*, Lawrence Erlbaum Associates, Hillsdale, N.J., 1986.
3. C. Riesbeck and R.C. Schank, *Inside Case-Based Reasoning*, Lawrence Erlbaum, 1989.
4. J.L. Kolodner, *Case-Based Reasoning*, Morgan Kaufmann, San Francisco, 1993.
5. P. Jackson, *Introduction to Expert Systems*, 2nd ed., Addison-Wesley, Reading, Mass., 1990.
6. P. Pu, "Special Issue: Case-Based Design Systems," *Artificial Intelligence in Engineering, Design and Manufacturing*, Cambridge Univ. Press, 1990.
7. M.L. Maher, B. Balachandran, and D.M. Zhang, *Case-Based Reasoning in Design*, Lawrence Erlbaum, 1995.
8. M.L. Maher and P. Pu, eds., *Issues and Applications of Case-Based Reasoning in Design*, Lawrence Erlbaum, 1997.
9. U. Flemming et al., "Case-Based Design in a Software Environment That Supports the Early Phases," in *Issues and Applications of Case-Based Reasoning in Design*, M.L. Maher and P. Pu, eds., Lawrence Erlbaum, 1997.

Mary Lou Maher is codirector of the Key Centre of Design Computing and an associate professor in the Department of Architectural and Design Science at the University of Sydney, Australia. Her research interests include case-based reasoning in design, modeling creative design using genetic algorithms, and computer support for collaborative design. She is the editor or coauthor of eight books and over 100 papers on design computing. She received her BS in civil engineering at Columbia University and her MS and PhD in civil engineering at Carnegie Mellon University. She can be contacted at the Dept. of Architectural and Design Science, Univ. of Sydney, NSW 2006, Australia; mary@arch.su.edu.au; http://www.arch.su.edu.au/~mary/.

Andrés Gómez de Silva Garza is a PhD student in the Department of Architectural and Design Science at the University of Sydney. His research interests include case-based reasoning, genetic algorithms, and design science. He received his BSc in computer engineering from the National Autonomous University of Mexico in 1991, and his MSc in computer science from the Georgia Institute of Technology in 1993. He is a member of the AAAI. He can be contacted at the Dept. of Architectural and Design Science, Univ. of Sydney, NSW 2006, Australia; andres@arch.su.edu.au; http://www.arch.su.edu.au/~andres/index.html.

Coming in June

定性推理

尽管——也有人认为正因如此——越来越多的复杂技术被用于对从机电装置到诸如股票市场等非物理系统的建模与分析，但对行为的定性理解以及运用定性知识进行推理的能力在人类智能与机器智能中依然至关重要。由斯坦福大学知识系统实验室的岩崎由美担任客座编辑，本期特刊将通过考察以下内容来记录该领域的最新进展：

•复杂工业过程的建模与仿真

•基于模型的序列控制程序自动生成方法，源自设计信息

•基于定性模型的燃气轮机状态监测

•利用定性物理学构建用于科学与工程教育的智能软件

•定性仿真与预测

•作为基于记忆的预测指标的定性模型

•美国国家航空航天局当前在自主航天器方面的研究工作

More AI News You Can Use

基于案例的设计系统激发了大量关于设计领域人工智能的研究。将案例推理作为统摄性框架，用以阐释如何将设计知识或经验的正式模型应用于新的设计问题，已变得十分普遍。这种混合方法的一个典型示例便是将基于案例的推理与基于模型的推理相结合。案例推理中有关记忆组织、索引和检索的概念可直接适用于基于模型的推理。在诸如Kritik之类的系统中，我们甚至可以看到两者之间的界限趋于模糊。案例推理范式的通用性表明，它能够被广泛应用于多种面向设计的人工智能方法。

最后，关于设计辅助或设计自动化的议题催生了两类基于案例的设计系统：一类为用户提供具备索引与检索功能的设计记忆库，另一类则能够自动检索或改编已有设计。这种区分反映了人们对用户（即人类设计师）角色的不同期望。在本研究对基于案例的设计系统的阐述中，这一区分并非核心问题。将基于案例推理应用于设计所引发的焦点在于设计案例的表示方式。设计案例记忆库的构建受到人工智能技术与设计研究的双重指导；而该记忆库在设计支持或设计自动化中的应用，则可能影响用户界面的设计以及算法的开发。

用于变更设计案例的算法，但并不改变对设计经验进行形式化表达的需求。

参考文献

1. R.C. Schank，《v0》，剑桥大学出版社，英国剑桥，1982年。 2. J.L. Kolodner 和 C.K. Riesbeck，《v1》，劳伦斯·埃尔姆鲍姆联合公司，新泽西州希尔斯代尔，1986年。 3. C. Riesbeck 和 R.C. Schank，《v2》，劳伦斯·埃尔姆鲍姆，1989年。 4. J.L. Kolodner，《v3》，摩根—考夫曼出版社，旧金山，1993年。 5. P. Jackson，《v4》，第二版，艾迪生-韦斯利出版社，马萨诸塞州雷丁，1990年。 6. P. Pu，“专题：基于案例的设计系统”，《v5》，剑桥大学出版社，1990年。 7. M.L. Maher、B. Balachandran 和 D.M. Zhang，《v6》，劳伦斯·埃尔姆鲍姆，1995年。 8. M.L. Maher 和 P. Pu 主编，《v7》，劳伦斯·埃尔姆鲍姆，1997年。

9. U. Flemming 等人，“在支持早期阶段的软件环境中开展基于案例的设计”，载于 M.L. Maher 和 P. Pu 主编，《第8卷》，劳伦斯·埃尔鲍姆出版社，1997年。

玛丽·卢·马赫尔是设计计算关键研究中心的联合主任，同时担任澳大利亚悉尼大学建筑与设计科学系的副教授。她的研究兴趣包括设计中的案例推理、利用遗传算法对创造性设计进行建模，以及计算机在协同设计中的支持。她是八部著作的主编或合著者，并发表了100余篇关于设计计算的论文。她于哥伦比亚大学获得土木工程学士学位，并在卡内基梅隆大学获得土木工程硕士和博士学位。可通过以下方式联系她：澳大利亚新南威尔士州悉尼大学建筑与设计科学系，邮编2006；电子邮箱：mary@arch.su.edu.au；网址：http://www.arch.su.edu.au/~mary/。

安德烈斯·戈麦斯·德·席尔瓦·加尔萨是悉尼大学建筑与设计科学系的博士研究生。他的研究兴趣包括基于案例的推理、遗传算法以及设计科学。他于1991年获得墨西哥国立自治大学计算机工程学士学位，1993年获得佐治亚理工学院计算机科学硕士学位。他是美国人工智能协会（AAAI）的会员。可通过以下方式与其联系：澳大利亚新南威尔士州悉尼大学建筑与设计科学系，邮编2006；电子邮箱：andres@arch.su.edu.au；网址：http://www.arch.su.edu.au/~andres/index.html。